

```
import pandas as pd

# Upload your Excel file
from google.colab import files

uploaded = files.upload()
```

EUREF_Po...del_EN.xlsx

EUREF_PowerBI_DataModel_EN.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 573234 bytes, last modified: 31/08/2026 - 100% done
Saving EUREF_PowerBI_DataModel_EN.xlsx to EUREF_PowerBI_DataModel_EN.xlsx

```
import pandas as pd

# Replace this with the exact filename shown after your upload
file_name = list(uploaded.keys())[0]

# Load the Excel workbook
excel_file = pd.ExcelFile(file_name)

# Display all sheet names
print("Sheets in the workbook:")
for i, sheet in enumerate(excel_file.sheet_names, 1):
    print(f"{i}. {sheet}")
```

```
Sheets in the workbook:
1. Fact_SurveyResponse
2. Fact_SurveyMultiSelect
3. Dim_LikertMap
4. Dim_Company
5. Fact_Veranstaltungen
6. Fact_Veranstaltungen_Venue
```

```
# Load all six sheets into separate DataFrames

df_survey = pd.read_excel(file_name, sheet_name="Fact_SurveyResponse")
df_multiselect = pd.read_excel(file_name, sheet_name="Fact_SurveyMultiSelect")
df_likert = pd.read_excel(file_name, sheet_name="Dim_LikertMap")
df_company = pd.read_excel(file_name, sheet_name="Dim_Company")
df_events = pd.read_excel(file_name, sheet_name="Fact_Veranstaltungen")
df_event_venue = pd.read_excel(file_name, sheet_name="Fact_Veranstaltungen_Venue")

# Show number of rows and columns in each sheet
print("DATASET SIZES")
print("-" * 50)

datasets = {
    "Fact_SurveyResponse": df_survey,
```

```
"Fact_SurveyMultiSelect": df_multiselect,
"Dim_LikertMap": df_likert,
"Dim_Company": df_company,
"Fact_Veranstaltungen": df_events,
"Fact_Veranstaltungen_Venue": df_event_venue
}

for name, df in datasets.items():
    print(f"{name}: {df.shape[0]:,} rows × {df.shape[1]} columns")
```

DATASET SIZES

```
-----
Fact_SurveyResponse: 120 rows × 34 columns
Fact_SurveyMultiSelect: 1,787 rows × 4 columns
Dim_LikertMap: 4 rows × 2 columns
Dim_Company: 6,148 rows × 4 columns
Fact_Veranstaltungen: 5,572 rows × 11 columns
Fact_Veranstaltungen_Venue: 6,168 rows × 2 columns
```

```
# Display all column names for each dataset
```

```
for name, df in datasets.items():
    print("\n" + "=" * 70)
    print(name)
    print("=" * 70)

    for i, column in enumerate(df.columns, 1):
        print(f"{i}. {column}")
```

```

=====
Dim_LikertMap
=====
1. Likert_EN
2. Score

=====
Dim_Company
=====
1. CompanyRowID
2. Company_Location
3. Industry
4. Campus

=====
Fact_Veranstaltungen
=====
1. EventRowID
2. Event_Title
3. Event_Date
4. Marketing_Measure
5. Occasion_Group
6. Campus
7. PAX
8. LastModified_ProxyDate
9. Cancellation_Category
10. IsRealEvent
11. IsCancelled

=====
Fact_Veranstaltungen_Venue
=====
1. EventRowID
2. Venue

```

```
# Inspect the key survey variables
```

```

columns_to_check = [
    "Org_Location",
    "Industry",
    "Most_Influential_Channel",
    "Booking_Mode",
    "Events_Per_Year",
    "Aware_of_EUREF"
]

for col in columns_to_check:
    print("\n" + "=" * 60)
    print(col)
    print("=" * 60)
    print(df_survey[col].value_counts(dropna=False))

```

```
Technology & Telecommunications    29
Energy & Sustainability            11
Consulting & Professional Services 11
Education                         11
Other                             11
Events, Hospitality & MICE         8
Financial Services                 8
Media, Marketing & Advertising     8
Mobility & Automotive              7
Manufacturing & Industry           7
Government / Public Sector         5
Healthcare                        3
Non-Profit                        1
Name: count, dtype: int64
```

```
=====
Most_Influential_Channel
=====
```

```
Most_Influential_Channel
Event agency                27
Search engines              24
Social media                17
Professional referrals      15
Existing venue relationship 13
Venue website               8
Direct outreach by venue    7
Online venue platforms      7
Trade fairs/events          2
Name: count, dtype: int64
```

```
=====
Booking_Mode
=====
```

```
Booking_Mode
Both, depending on event    48
Direct with venue           32
Via event agency            31
Other                       9
Name: count, dtype: int64
```

```

Yes, but never considered it      23
Knew EUREF Campus, not its event spaces  18
No, was not aware                  13
Name: count, dtype: int64

```

```
# Inspect all Likert/satisfaction score variables
```

```

score_columns = [
    "Initial Response to Enquiry_Score",
    "Communication During Planning_Score",
    "Booking Process_Score",
    "Price Transparency_Score",
    "Technical Facilities_Score",
    "Internet Connection_Score",
    "Catering_Score",
    "Staff Support_Score",
    "Accessibility_Score",
    "Flexibility & Customisation_Score",
    "Sustainability Offering_Score",
    "Overall Experience_Score",
    "Avg_Satisfaction_Score"
]

for col in score_columns:
    print("\n" + "=" * 60)
    print(col)
    print("=" * 60)
    print("Unique values:", sorted(df_survey[col].dropna().unique()))
    print("Missing values:", df_survey[col].isna().sum())
    print("Counts:")
    print(df_survey[col].value_counts(dropna=False).sort_index())

```

```
=====
Avg_Satisfaction_Score
=====
Unique values: [np.float64(1.272727272727273), np.float64(1.333333333333333), np.float64(1.5), np.float64(1.583333333333333), np.float64(1.777777777777778)]
Missing values: 21
Counts:
Avg_Satisfaction_Score
1.272727      1
1.333333      1
1.500000      2
1.583333      1
1.777778      1
1.818182      1
1.833333      1
1.916667      1
2.000000     14
2.083333      1
2.090909      2
2.166667      5
2.250000      5
2.272727      1
2.333333      4
2.363636      2
2.416667      3
2.454545      1
2.500000      6
2.545455      2
2.583333      8
2.666667      3
2.714286      1
2.750000      2
2.833333      1
2.916667      4
3.000000     25
NaN           21
Name: count, dtype: int64
```

```
# Display the Likert scale mapping
```

```
print(df_likert)
```

	Likert_EN	Score
0	Dissatisfied	1.0
1	Neutral	2.0
2	Satisfied	3.0
3	Not sure	NaN

```
# Inspect the different multi-select questions
```

```
print("Number of unique questions:", df_multiselect["QuestionKey"].nunique())
```

```
print("\nQuestion keys and labels:")
print(
    df_multiselect[["QuestionKey", "QuestionLabel"]]
    .drop_duplicates()
    .to_string(index=False)
)
```

Number of unique questions: 5

Question keys and labels:

QuestionKey	QuestionLabel
discovery_channel	How organisation typically finds venues
agency_reasons	Main reasons for using event agencies
direct_booking_incentive	What would encourage booking directly
decision_factors	Most important venue selection factors
euref_future_incentive	What would increase likelihood of choosing EUREF

Show response counts for each multi-select question

```
for key in df_multiselect["QuestionKey"].unique():
    print("\n" + "=" * 70)
    print(f"Question: {key}")
    print("=" * 70)

    temp = df_multiselect[df_multiselect["QuestionKey"] == key]

    print(temp["Answer"].value_counts(dropna=False))
```

```
=====
Question: decision_factors
=====
Answer
Location & accessibility      85
Venue capacity                63
Value for money               58
Technical/AV equipment        53
Fast communication & response time 49
Flexible event spaces         36
Sustainability features       32
Venue reputation              27
Past experience with venue     26
Name: count, dtype: int64

=====
Question: euref_future_incentive
=====
Answer
More competitive pricing      55
Faster response times         49
Simpler enquiry/booking process 41
Tenant discounts              40
Better communication on available spaces 37
Clearer pricing information    35
More flexible contracts        34
Dedicated point of contact     34
Customised event packages      32
Better technical/hybrid event options 27
Stronger sustainability packages 20
More information on event services 14
Other                          10
Name: count, dtype: int64
```

```
# Check the number of selections made by each respondent
# for each multi-select question

for key in df_multiselect["QuestionKey"].unique():

    temp = df_multiselect[df_multiselect["QuestionKey"] == key]

    selections_per_respondent = (
        temp.groupby("ResponseID")["Answer"]
        .count()
    )

    print("\n" + "=" * 70)
    print(f"Question: {key}")
    print("=" * 70)
    print("Respondents:", selections_per_respondent.shape[0])
    print("Average selections per respondent:", round(selections_per_respondent.mean(), 2))
    print("Minimum selections:", selections_per_respondent.min())
    print("Maximum selections:", selections_per_respondent.max())
```

```
print("\nDistribution of number of selections:")  
print(selections_per_respondent.value_counts().sort_index())
```

Distribution of number of selections:

Answer

1	38
2	12
3	17
4	9
5	16

```

/      /
8      3
9      3
10     2
12     3
13     1
Name: count, dtype: int64

```

```
# Inspect the key CRM variables for RQ1
```

```

crm_columns = [
    "Marketing_Measure",
    "Cancellation_Category",
    "IsCancelled",
    "IsRealEvent"
]

for col in crm_columns:
    print("\n" + "=" * 60)
    print(col)
    print("=" * 60)
    print(df_events[col].value_counts(dropna=False))

```

```

=====
Marketing_Measure
=====
Marketing_Measure
Operational          1525
Other                1156
Recurring             877
Campus Tenant        599
Digital              416
Agency              261
Network              241
Unclassified         219
Internal             166
Industry Association   65
Venue Portal         47
Name: count, dtype: int64

```

```

=====
Cancellation_Category
=====
Cancellation_Category
NaN                  3439
Other                196
Customer - Event Not Taking Place  190
Customer - Total Price Too High    173
Test Booking          163
Other Venue - Budget    150
EUREF - Capacity       147
Customer - Ambience    140
EUREF Cancelled - Capacity  139

```

Chose Other Venue	108
Other Venue – Ambience	98
Event Not Taking Place	87
EUREF Cancelled – Budget	57
Customer – Lost Pitch	55
EUREF – Budget	49
Customer – Event Not in Berlin	45
Lost Pitch	43
BP Test Booking	32
Budget Too Small	29
No Response to Offer	28
Event Not in Berlin	26
Alternative Date Chosen	22
Multiple Venues Requested	19
Customer – Chose Other Provider	17
Customer – Date Not Available	16
Chose Other City	15
Cancelled by Us	14
Customer – No Response	10
Event No Longer Taking Place	8
EUREF Internal Room Block	6
Customer – Chose Alternative Date	6
Customer – Overnight Accommodation	5
No Decision Possible Yet	4
Rooms Too Small	4
Customer – Offer Wait Time Too Long	4
EUREF – Exceeds Room Capacity	4

```
# Check real vs non-real events and their cancellation status
```

```
print("IsRealEvent × IsCancelled")
print("=" * 50)

cross_real_cancel = pd.crosstab(
    df_events["IsRealEvent"],
    df_events["IsCancelled"],
    margins=True
)

print(cross_real_cancel)

print("\nPercent cancelled within each event type:")
print(
    pd.crosstab(
        df_events["IsRealEvent"],
        df_events["IsCancelled"],
        normalize="index"
    ).round(3) * 100
)
```

```
IsRealEvent × IsCancelled
=====
IsCancelled False True All
```

```

IsRealEvent
False      1726   128  1854
True       1713  2005  3718
All        3439  2133  5572

```

Percent cancelled within each event type:

```

IsCancelled  False  True
IsRealEvent
False        93.1   6.9
True         46.1  53.9

```

Check lead-source distribution by real/non-real event status

```

lead_real_table = pd.crosstab(
    df_events["Marketing_Measure"],
    df_events["IsRealEvent"],
    margins=True
)

print(lead_real_table)

print("\nPercentage of real events within each lead source:")
print(
    pd.crosstab(
        df_events["Marketing_Measure"],
        df_events["IsRealEvent"],
        normalize="index"
    ).round(3) * 100
)

```

```

IsRealEvent      False  True  All
Marketing_Measure
Agency           12    249  261
Campus Tenant     24    575  599
Digital           18    398  416
Industry Association  2     63   65
Internal          62    104  166
Network           6    235  241
Operational      1517     8  1525
Other             91   1065  1156
Recurring         32    845  877
Unclassified      88    131  219
Venue Portal       2     45   47
All              1854   3718  5572

```

Percentage of real events within each lead source:

```

IsRealEvent      False  True
Marketing_Measure
Agency           4.6   95.4
Campus Tenant     4.0   96.0
Digital           4.3   95.7
Industry Association  3.1   96.9

```

Internal	37.3	62.7
Network	2.5	97.5
Operational	99.5	0.5
Other	7.9	92.1
Recurring	3.6	96.4
Unclassified	40.2	59.8
Venue Portal	4.3	95.7

```
# =====
# RQ1 DATASET – MATCHING THE UPDATED POWER BI DASHBOARD
# =====

# Use real events only
df_rq1 = df_events[df_events["IsRealEvent"] == True].copy()

# Lead sources shown in the updated Power BI dashboard
lead_sources = [
    "Operational",
    "Recurring",
    "Campus Tenant",
    "Digital",
    "Agency",
    "Network",
    "Internal",
    "Industry Association",
    "Venue Portal"
]

# Keep only these lead-source categories
df_rq1 = df_rq1[
    df_rq1["Marketing_Measure"].isin(lead_sources)
].copy()

print("RQ1 DATASET")
print("=" * 60)
print("Total real events:", len(df_rq1))

print("\nLead-source distribution:")
print(df_rq1["Marketing_Measure"].value_counts())

print("\nBooking outcome distribution:")
print(df_rq1["IsCancelled"].value_counts())

print("\nLead Source × Booking Outcome")
print("=" * 60)

rq1_crosstab = pd.crosstab(
    df_rq1["Marketing_Measure"],
    df_rq1["IsCancelled"]
)
```

```
print(rq1_crosstab)
```

```
RQ1 DATASET
```

```
=====
```

```
Total real events: 2522
```

```
Lead-source distribution:
```

```
Marketing_Measure
```

Recurring	845
Campus Tenant	575
Digital	398
Agency	249
Network	235
Internal	104
Industry Association	63
Venue Portal	45
Operational	8

```
Name: count, dtype: int64
```

```
Booking outcome distribution:
```

```
IsCancelled
```

True	1421
False	1101

```
Name: count, dtype: int64
```

```
Lead Source × Booking Outcome
```

```
=====
```

IsCancelled	False	True
Marketing_Measure		
Agency	32	217
Campus Tenant	387	188
Digital	88	310
Industry Association	46	17
Internal	76	28
Network	105	130
Operational	6	2
Recurring	348	497
Venue Portal	13	32

```
# =====
```

```
# RQ1 – CHI-SQUARE TEST OF INDEPENDENCE
```

```
# Lead Source × Booking Outcome
```

```
# =====
```

```
from scipy.stats import chi2_contingency
```

```
# Create contingency table
```

```
contingency_table = pd.crosstab(
```

```
    df_rq1["Marketing_Measure"],
```

```
    df_rq1["IsCancelled"]
```

```
)
```

```

print("CONTINGENCY TABLE")
print("=" * 60)
print(contingency_table)

# Perform Chi-square test
chi2, p_value, degrees_of_freedom, expected = chi2_contingency(
    contingency_table
)

print("\nCHI-SQUARE TEST RESULTS")
print("=" * 60)
print(f"Chi-square statistic: {chi2:.4f}")
print(f"Degrees of freedom: {degrees_of_freedom}")
print(f"P-value: {p_value:.6f}")

# Statistical decision
alpha = 0.05

print("\nSTATISTICAL DECISION")
print("=" * 60)

if p_value < alpha:
    print("Result: Statistically significant")
    print("Reject the null hypothesis (H0).")
    print("Lead source and booking outcome are associated.")
else:
    print("Result: Not statistically significant")
    print("Fail to reject the null hypothesis (H0).")
    print("No statistically significant association was detected.")

```

CONTINGENCY TABLE

```

=====
IsCancelled      False  True
Marketing_Measure
Agency           32    217
Campus Tenant    387    188
Digital           88    310
Industry Association  46    17
Internal          76    28
Network          105   130
Operational        6     2
Recurring         348   497
Venue Portal      13    32

```

CHI-SQUARE TEST RESULTS

```

=====
Chi-square statistic: 369.9619
Degrees of freedom: 8
P-value: 0.000000

```

STATISTICAL DECISION

Result: Statistically significant
 Reject the null hypothesis (H_0).
 Lead source and booking outcome are associated.

```
# =====
# RQ1 – CRAMÉR'S V
# Strength of association between Lead Source and Booking Outcome
# =====

import numpy as np

# Calculate Cramér's V
n = contingency_table.sum().sum()
min_dimension = min(contingency_table.shape) - 1

cramers_v = np.sqrt(
    chi2 / (n * min_dimension)
)

print("CRAMÉR'S V")
print("=" * 60)
print(f"Sample size (N): {n}")
print(f"Cramér's V: {cramers_v:.4f}")

# Interpretation
print("\nINTERPRETATION")
print("=" * 60)

if crammers_v < 0.10:
    print("Very weak/negligible association")
elif crammers_v < 0.30:
    print("Weak association")
elif crammers_v < 0.50:
    print("Moderate association")
else:
    print("Strong association")
```

```
CRAMÉR'S V
=====
Sample size (N): 2522
Cramér's V: 0.3830

INTERPRETATION
=====
Moderate association
```

```
# =====
# RQ1 – STANDARDISED RESIDUALS
# Identify which lead sources drive the association
# =====
```

```

# Expected frequencies
expected = pd.DataFrame(
    chi2_contingency(contingency_table)[3],
    index=contingency_table.index,
    columns=contingency_table.columns
)

# Standardised residuals
residuals = (contingency_table - expected) / np.sqrt(expected)

print("EXPECTED FREQUENCIES")
print("=" * 60)
print(expected.round(2))

print("\nSTANDARDISED RESIDUALS")
print("=" * 60)
print(residuals.round(2))

print("\nINTERPRETATION")
print("=" * 60)
print("Residuals above +1.96 indicate more cases than expected.")
print("Residuals below -1.96 indicate fewer cases than expected.")
print("Values between -1.96 and +1.96 are not statistically unusual.")

```

EXPECTED FREQUENCIES

	False	True
IsCancelled		
Marketing_Measure		
Agency	108.70	140.30
Campus Tenant	251.02	323.98
Digital	173.75	224.25
Industry Association	27.50	35.50
Internal	45.40	58.60
Network	102.59	132.41
Operational	3.49	4.51
Recurring	368.89	476.11
Venue Portal	19.65	25.35

STANDARDISED RESIDUALS

	False	True
IsCancelled		
Marketing_Measure		
Agency	-7.36	6.48
Campus Tenant	8.58	-7.55
Digital	-6.51	5.73
Industry Association	3.53	-3.10
Internal	4.54	-4.00
Network	0.24	-0.21
Operational	1.34	-1.18
Recurring	-1.09	0.96
Venue Portal	-1.50	1.32

INTERPRETATION

=====

Residuals above +1.96 indicate more cases than expected.
 Residuals below -1.96 indicate fewer cases than expected.
 Values between -1.96 and +1.96 are not statistically unusual.

```
# =====
# RQ1 - CANCELLATION RATE BY LEAD SOURCE
# =====

rq1_rates = (
    df_rq1
    .groupby("Marketing_Measure")["IsCancelled"]
    .agg(
        Total_Events="count",
        Cancelled="sum"
    )
    .reset_index()
)

# Calculate non-cancelled events
rq1_rates["Not_Cancelled"] = (
    rq1_rates["Total_Events"] - rq1_rates["Cancelled"]
)

# Calculate percentages
rq1_rates["Cancellation_Rate_%"] = (
    rq1_rates["Cancelled"] /
    rq1_rates["Total_Events"] * 100
)

rq1_rates["Booking_Rate_%"] = (
    rq1_rates["Not_Cancelled"] /
    rq1_rates["Total_Events"] * 100
)

# Sort from highest to lowest cancellation rate
rq1_rates = rq1_rates.sort_values(
    "Cancellation_Rate_%",
    ascending=False
)

print(rq1_rates.round(2).to_string(index=False))
```

Marketing_Measure	Total_Events	Cancelled	Not_Cancelled	Cancellation_Rate_%	Booking_Rate_%
Agency	249	217	32	87.15	12.85
Digital	398	310	88	77.89	22.11
Venue Portal	45	32	13	71.11	28.89
Recurring	845	497	348	58.82	41.18

Network	235	130	105	55.32	44.68
Campus Tenant	575	188	387	32.70	67.30
Industry Association	63	17	46	26.98	73.02
Internal	104	28	76	26.92	73.08
Operational	8	2	6	25.00	75.00

```
# =====
# RQ1 – 95% CONFIDENCE INTERVALS FOR CANCELLATION RATES
# =====
```

```
from statsmodels.stats.proportion import proportion_confint
```

```
rq1_ci = rq1_rates.copy()
```

```
# Calculate 95% Wilson confidence intervals
```

```
ci = rq1_ci.apply(
    lambda row: proportion_confint(
        count=int(row["Cancelled"]),
        nobs=int(row["Total_Events"]),
        alpha=0.05,
        method="wilson"
    ),
    axis=1
)
```

```
rq1_ci["CI_Lower_%"] = [x[0] * 100 for x in ci]
rq1_ci["CI_Upper_%"] = [x[1] * 100 for x in ci]
```

```
print(
    rq1_ci[
        [
            "Marketing_Measure",
            "Total_Events",
            "Cancelled",
            "Cancellation_Rate_%",
            "CI_Lower_%",
            "CI_Upper_%"
        ]
    ].round(2).to_string(index=False)
)
```

Marketing_Measure	Total_Events	Cancelled	Cancellation_Rate_%	CI_Lower_%	CI_Upper_%
Agency	249	217	87.15	82.42	90.75
Digital	398	310	77.89	73.56	81.69
Venue Portal	45	32	71.11	56.63	82.27
Recurring	845	497	58.82	55.47	62.09
Network	235	130	55.32	48.93	61.54
Campus Tenant	575	188	32.70	28.99	36.63
Industry Association	63	17	26.98	17.58	39.03
Internal	104	28	26.92	19.33	36.16
Operational	8	2	25.00	7.15	59.07

done Rq 1

```
# =====
# RQ2 – CREATE RESPONDENT × DECISION-FACTOR MATRIX
# =====

# Select the decision-factor responses
df_rq2_multi = df_multiselect[
    df_multiselect["QuestionKey"] == "decision_factors"
].copy()

# Create binary respondent × factor matrix
rq2_matrix = pd.crosstab(
    df_rq2_multi["ResponseID"],
    df_rq2_multi["Answer"]
)

# Convert to binary: selected = 1, not selected = 0
rq2_matrix = (rq2_matrix > 0).astype(int)

print("RQ2 MATRIX")
print("=" * 60)
print("Respondents:", rq2_matrix.shape[0])
print("Factors:", rq2_matrix.shape[1])

print("\nNumber of respondents selecting each factor:")
print(rq2_matrix.sum().sort_values(ascending=False))

print("\nFirst 5 respondents:")
print(rq2_matrix.head())
```

RQ2 MATRIX

```
=====
Respondents: 120
Factors: 9
```

Number of respondents selecting each factor:

```
Answer
Location & accessibility      85
Venue capacity                63
Value for money               58
Technical/AV equipment        53
Fast communication & response time  49
Flexible event spaces         36
Sustainability features       32
Venue reputation              27
Past experience with venue     26
dtype: int64
```

First 5 respondents:

Answer ResponseID	Fast communication & response time	Flexible event spaces	\
1	1	0	
2	0	0	
3	1	0	
4	0	1	
5	0	0	

Answer ResponseID	Location & accessibility	Past experience with venue	\
1	1	1	
2	1	0	
3	1	0	
4	0	0	
5	1	0	

Answer ResponseID	Sustainability features	Technical/AV equipment	Value for money	\
1	0	0	1	
2	0	0	1	
3	0	0	1	
4	0	0	0	
5	0	0	0	

Answer ResponseID	Venue capacity	Venue reputation
1	1	1
2	0	0
3	1	0
4	0	0
5	0	0

```
# =====
# RQ2 – CHECK FOR MISSING VALUES
# =====

print("Missing values in RQ2 matrix:")
print(rq2_matrix.isna().sum())

print("\nTotal missing values:", rq2_matrix.isna().sum().sum())

print("\nData type of variables:")
print(rq2_matrix.dtypes)
```

Missing values in RQ2 matrix:

Answer	
Fast communication & response time	0
Flexible event spaces	0
Location & accessibility	0
Past experience with venue	0
Sustainability features	0
Technical/AV equipment	0

```

Value for money          0
Venue capacity           0
Venue reputation         0
dtype: int64

Total missing values: 0

Data type of variables:
Answer
Fast communication & response time    int64
Flexible event spaces                 int64
Location & accessibility              int64
Past experience with venue            int64
Sustainability features              int64
Technical/AV equipment               int64
Value for money                      int64
Venue capacity                      int64
Venue reputation                    int64
dtype: object

```

```

# =====
# RQ2 – COCHRAN'S Q TEST
# Venue Selection Factors
# =====

# Install Pingouin if it is not already installed
!pip -q install pingouin

import pandas as pd
import numpy as np
import pingouin as pg

# -----
# 1. Check the RQ2 matrix
# -----

print("RQ2 DATA CHECK")
print("=" * 60)

print(f"Number of respondents: {rq2_matrix.shape[0]}")
print(f"Number of venue-selection factors: {rq2_matrix.shape[1]}")
print(f"Total missing values: {rq2_matrix.isna().sum().sum()}")

print("\nSelection counts:")
print(rq2_matrix.sum().sort_values(ascending=False))

# -----
# 2. Run Cochran's Q test
# -----

cochran_result = pg.cochran(

```

```

cochran_result = pg.cochran(
    data=rq2_matrix
)

print("\n" + "=" * 60)
print("COCHRAN'S Q TEST")
print("=" * 60)

print(cochran_result)

# -----
# 3. Extract results
# -----

Q = cochran_result["Q"].iloc[0]
p = cochran_result["p_unc"].iloc[0]
df = cochran_result["dof"].iloc[0]

# -----
# 4. Print clean statistical result
# -----

print("\n" + "=" * 60)
print("STATISTICAL RESULT")
print("=" * 60)

print(f"Cochran's Q = {Q:.3f}")
print(f"Degrees of freedom = {int(df)}")
print(f"P-value = {p:.3e}")

if p < 0.05:
    print("\nResult: STATISTICALLY SIGNIFICANT")
    print("Decision: Reject H0.")
    print(
        "Conclusion: The nine venue-selection factors "
        "were not selected at equal rates."
    )
else:
    print("\nResult: NOT STATISTICALLY SIGNIFICANT")
    print("Decision: Fail to reject H0.")
    print(
        "Conclusion: No statistically significant difference "
        "was detected between the selection rates."
    )

# -----
# 5. Display selection percentages
# -----

```

```

selection_summary = pd.DataFrame({
    "Factor": rq2_matrix.columns,
    "Selected": rq2_matrix.sum().values,
    "Not_Selected": (1 - rq2_matrix).sum().values
})

selection_summary["Selection_Rate_%"] = (
    selection_summary["Selected"]
    / len(rq2_matrix)
    * 100
)

selection_summary = selection_summary.sort_values(
    "Selection_Rate_%",
    ascending=False
)

print("\n" + "=" * 60)
print("VENUE-SELECTION FACTOR RANKING")
print("=" * 60)

print(
    selection_summary.round(2).to_string(index=False)
)

```

RQ2 DATA CHECK

```

=====
Number of respondents: 120
Number of venue-selection factors: 9
Total missing values: 0

```

Selection counts:

```

Answer
Location & accessibility      85
Venue capacity                63
Value for money               58
Technical/AV equipment        53
Fast communication & response time 49
Flexible event spaces         36
Sustainability features      32
Venue reputation              27
Past experience with venue    26
dtype: int64

```

COCHRAN'S Q TEST

```

=====
              Source  dof          Q      p_unc
cochran  Within      8  124.527273  3.844912e-23

```

STATISTICAL RESULT

```
=====
Cochran's Q = 124.527
Degrees of freedom = 8
P-value = 3.845e-23
```

Result: STATISTICALLY SIGNIFICANT

Decision: Reject H_0 .

Conclusion: The nine venue-selection factors were not selected at equal rates.

VENUE-SELECTION FACTOR RANKING

```
=====
```

	Factor	Selected	Not_Selected	Selection_Rate_%
	Location & accessibility	85	35	70.83
	Venue capacity	63	57	52.50
	Value for money	58	62	48.33
	Technical/AV equipment	53	67	44.17
Fast communication & response time		49	71	40.83
	Flexible event spaces	36	84	30.00
	Sustainability features	32	88	26.67
	Venue reputation	27	93	22.50
Past experience with venue		26	94	21.67

```
=====
```

```
# =====
# RQ2 – POST-HOC MCNEMAR TESTS
# Pairwise comparison of venue-selection factors
# Bonferroni correction applied
# =====

from itertools import combinations
from statsmodels.stats.contingency_tables import mcnemar
from statsmodels.stats.multitest import multipletests

# Store results
posthoc_results = []

# Get all factors
factors = rq2_matrix.columns.tolist()

# Number of pairwise comparisons
num_comparisons = len(list(combinations(factors, 2)))

print("Number of pairwise comparisons:", num_comparisons)

# -----
# Run pairwise McNemar tests
# -----

for factor1, factor2 in combinations(factors, 2):
```

```
# 2 × 2 table:
#           Factor 2
#           0      1
# Factor 1
# 0
# 1
table = pd.crosstab(
    rq2_matrix[factor1],
    rq2_matrix[factor2]
)

# Ensure 2 × 2 structure
table = table.reindex(
    index=[0, 1],
    columns=[0, 1],
    fill_value=0
)

# McNemar test
result = mcnemar(
    table,
    exact=True
)

posthoc_results.append({
    "Factor_1": factor1,
    "Factor_2": factor2,
    "Chi2": result.statistic,
    "P_value": result.pvalue
})

# Convert results to DataFrame
posthoc_df = pd.DataFrame(posthoc_results)

# -----
# Bonferroni correction
# -----

reject, p_adjusted, _, _ = multipletests(
    posthoc_df["P_value"],
    alpha=0.05,
    method="bonferroni"
)

posthoc_df["Adjusted_P"] = p_adjusted
posthoc_df["Significant"] = reject

# Sort by adjusted p-value
posthoc_df = posthoc_df.sort_values(
    "Adjusted_P"
```

```
)

# -----
# Display all results
# -----

print("\n" + "=" * 80)
print("PAIRWISE MCNEMAR TESTS WITH BONFERRONI CORRECTION")
print("=" * 80)

print(
    posthoc_df[
        [
            "Factor_1",
            "Factor_2",
            "Chi2",
            "P_value",
            "Adjusted_P",
            "Significant"
        ]
    ].round(4).to_string(index=False)
)

# -----
# Display only statistically significant comparisons
# -----

significant_pairs = posthoc_df[
    posthoc_df["Significant"] == True
]

print("\n" + "=" * 80)
print("STATISTICALLY SIGNIFICANT PAIRWISE DIFFERENCES")
print("=" * 80)

if len(significant_pairs) > 0:
    print(
        significant_pairs[
            [
                "Factor_1",
                "Factor_2",
                "Adjusted_P"
            ]
        ].round(4).to_string(index=False)
    )
else:
    print("No statistically significant pairwise differences were found.")
```

Number of pairwise comparisons: 36

PAIRWISE MCNEMAR TESTS WITH BONFERRONI CORRECTION						
	Factor_1	Factor_2	Chi2	P_value	Adjusted_P	Significant
	Location & accessibility	Past experience with venue	3.0	0.0000	0.0000	True
	Location & accessibility	Venue reputation	4.0	0.0000	0.0000	True
	Location & accessibility	Sustainability features	7.0	0.0000	0.0000	True
	Flexible event spaces	Location & accessibility	11.0	0.0000	0.0000	True
Fast communication & response time	Location & accessibility	Location & accessibility	7.0	0.0000	0.0000	True
	Past experience with venue	Venue capacity	8.0	0.0000	0.0000	True
	Venue capacity	Venue reputation	11.0	0.0000	0.0001	True
	Location & accessibility	Technical/AV equipment	9.0	0.0000	0.0002	True
	Past experience with venue	Value for money	10.0	0.0000	0.0003	True
	Value for money	Venue reputation	10.0	0.0000	0.0005	True
	Sustainability features	Venue capacity	12.0	0.0000	0.0012	True
	Past experience with venue	Technical/AV equipment	10.0	0.0001	0.0035	True
	Technical/AV equipment	Venue reputation	9.0	0.0001	0.0038	True
	Location & accessibility	Value for money	11.0	0.0001	0.0051	True
	Sustainability features	Value for money	14.0	0.0005	0.0193	True
Fast communication & response time	Past experience with venue	Past experience with venue	11.0	0.0008	0.0297	True
	Location & accessibility	Venue capacity	10.0	0.0009	0.0339	True
	Flexible event spaces	Venue capacity	19.0	0.0011	0.0392	True
Fast communication & response time		Venue reputation	11.0	0.0013	0.0454	True
	Sustainability features	Technical/AV equipment	10.0	0.0015	0.0522	False
	Flexible event spaces	Value for money	17.0	0.0046	0.1642	False
Fast communication & response time		Sustainability features	10.0	0.0076	0.2748	False
	Flexible event spaces	Technical/AV equipment	18.0	0.0270	0.9723	False
Fast communication & response time		Flexible event spaces	16.0	0.0725	1.0000	False
	Flexible event spaces	Venue reputation	15.0	0.1996	1.0000	False
	Flexible event spaces	Sustainability features	16.0	0.6177	1.0000	False
	Flexible event spaces	Past experience with venue	17.0	0.1742	1.0000	False
	Past experience with venue	Sustainability features	18.0	0.4408	1.0000	False
	Past experience with venue	Venue reputation	16.0	1.0000	1.0000	False
Fast communication & response time		Technical/AV equipment	15.0	0.6076	1.0000	False
Fast communication & response time		Value for money	17.0	0.2221	1.0000	False
Fast communication & response time		Venue capacity	16.0	0.0541	1.0000	False
	Technical/AV equipment	Venue capacity	17.0	0.1742	1.0000	False
	Technical/AV equipment	Value for money	23.0	0.5758	1.0000	False
	Sustainability features	Venue reputation	15.0	0.4996	1.0000	False
	Value for money	Venue capacity	24.0	0.5831	1.0000	False

STATISTICALLY SIGNIFICANT PAIRWISE DIFFERENCES

	Factor_1	Factor_2	Adjusted_P
	Location & accessibility	Past experience with venue	0.0000
	Location & accessibility	Venue reputation	0.0000
	Location & accessibility	Sustainability features	0.0000
	Flexible event spaces	Location & accessibility	0.0000
Fast communication & response time	Location & accessibility	Location & accessibility	0.0000
	Past experience with venue	Venue capacity	0.0000
	Venue capacity	Venue reputation	0.0001
	Location & accessibility	Technical/AV equipment	0.0002
	Past experience with venue	Value for money	0.0003
	Value for money	Venue reputation	0.0005

```
# =====
# RQ2 – CHECK RESPONDENT GROUP SIZES BY ORGANISATION LOCATION
# =====

# Add organisation location to the RQ2 matrix
rq2_with_location = rq2_matrix.join(
    df_survey.set_index("ResponseID")["Org_Location"]
)

print("ORGANISATION LOCATION GROUPS")
print("=" * 60)

print(
    rq2_with_location["Org_Location"]
    .value_counts()
)

print("\nCross-check:")
print(
    pd.crosstab(
        rq2_with_location["Org_Location"],
        columns="Respondents"
    )
)
```

ORGANISATION LOCATION GROUPS

```
=====
Org_Location
Berlin                56
Outside Europe        47
Elsewhere in Germany  12
Elsewhere in Europe   5
Name: count, dtype: int64
```

```
Cross-check:
col_0      Respondents
Org_Location
Berlin                56
Elsewhere in Europe   5
Elsewhere in Germany  12
Outside Europe        47
```

```
# =====
# RQ3 – CUSTOMER INCENTIVES FOR STRATEGIC SALES DECISIONS
# =====

# -----
# 1. Direct-booking incentives
# -----
```

```
df_direct = df_multiselect[
    df_multiselect["QuestionKey"] == "direct_booking_incentive"
].copy()

rq3_direct_matrix = pd.crosstab(
    df_direct["ResponseID"],
    df_direct["Answer"]
)

rq3_direct_matrix = (rq3_direct_matrix > 0).astype(int)

# -----
# 2. EUREF future-choice incentives
# -----

df_future = df_multiselect[
    df_multiselect["QuestionKey"] == "euref_future_incentive"
].copy()

rq3_future_matrix = pd.crosstab(
    df_future["ResponseID"],
    df_future["Answer"]
)

rq3_future_matrix = (rq3_future_matrix > 0).astype(int)

# -----
# 3. Display direct-booking results
# -----

print("=" * 70)
print("DIRECT-BOOKING INCENTIVES")
print("=" * 70)

print("Respondents:", rq3_direct_matrix.shape[0])
print("Factors:", rq3_direct_matrix.shape[1])

direct_summary = pd.DataFrame({
    "Factor": rq3_direct_matrix.columns,
    "Selected": rq3_direct_matrix.sum().values
})

direct_summary["Selection_Rate_%"] = (
    direct_summary["Selected"] /
    len(rq3_direct_matrix) * 100
)

direct_summary = direct_summary.sort_values(
```

```

        "Selection_Rate_%",
        ascending=False
    )

    print(
        direct_summary.round(2).to_string(index=False)
    )

# -----
# 4. Display future EUREF incentives
# -----

print("\n" + "=" * 70)
print("EUREF FUTURE-CHOICE INCENTIVES")
print("=" * 70)

print("Respondents:", rq3_future_matrix.shape[0])
print("Factors:", rq3_future_matrix.shape[1])

future_summary = pd.DataFrame({
    "Factor": rq3_future_matrix.columns,
    "Selected": rq3_future_matrix.sum().values
})

future_summary["Selection_Rate_%"] = (
    future_summary["Selected"] /
    len(rq3_future_matrix) * 100
)

future_summary = future_summary.sort_values(
    "Selection_Rate_%",
    ascending=False
)

print(
    future_summary.round(2).to_string(index=False)
)

```

```

=====
DIRECT-BOOKING INCENTIVES
=====
Respondents: 120
Factors: 10

```

	Factor	Selected	Selection_Rate_%
	Transparent pricing	64	53.33
	Faster response times	62	51.67
	Simpler enquiry/booking process	45	37.50
	Dedicated point of contact	44	36.67
	Competitive pricing	40	33.33
	Better online information	39	32.50

Strong venue expertise	39	32.50
Better technical/event support	36	30.00
Customised event packages	34	28.33
Existing relationship with venue	25	20.83

=====

EUREF FUTURE-CHOICE INCENTIVES

=====

Respondents: 120

Factors: 13

	Factor	Selected	Selection_Rate_%
	More competitive pricing	55	45.83
	Faster response times	49	40.83
	Simpler enquiry/booking process	41	34.17
	Tenant discounts	40	33.33
Better communication on available spaces		37	30.83
	Clearer pricing information	35	29.17
	More flexible contracts	34	28.33
	Dedicated point of contact	34	28.33
	Customised event packages	32	26.67
Better technical/hybrid event options		27	22.50
	Stronger sustainability packages	20	16.67
	More information on event services	14	11.67
	Other	10	8.33

```
# =====
# RQ3 - VALIDATE CUSTOMER INCENTIVE MATRICES
# =====

print("=" * 70)
print("RQ3 DATA VALIDATION")
print("=" * 70)

# Direct-booking incentives
print("\nDIRECT-BOOKING INCENTIVES")
print("-" * 50)
print("Respondents:", rq3_direct_matrix.shape[0])
print("Factors:", rq3_direct_matrix.shape[1])
print("Missing values:", rq3_direct_matrix.isna().sum().sum())
print("Duplicate ResponseIDs:", rq3_direct_matrix.index.duplicated().sum())

# Future EUREF incentives
print("\nEUREF FUTURE-CHOICE INCENTIVES")
print("-" * 50)
print("Respondents:", rq3_future_matrix.shape[0])
print("Factors:", rq3_future_matrix.shape[1])
print("Missing values:", rq3_future_matrix.isna().sum().sum())
print("Duplicate ResponseIDs:", rq3_future_matrix.index.duplicated().sum())

# Compare respondent IDs between the two datasets
direct_ids = set(rq3_direct_matrix.index)
```

```

future_ids = set(rq3_future_matrix.index)

print("\nRESPONDENT ID COMPARISON")
print("-" * 50)
print("Respondents in both datasets:", len(direct_ids & future_ids))
print("Only in direct-booking dataset:", len(direct_ids - future_ids))
print("Only in future-choice dataset:", len(future_ids - direct_ids))

```

```

=====
RQ3 DATA VALIDATION
=====

```

DIRECT-BOOKING INCENTIVES

```

-----
Respondents: 120
Factors: 10
Missing values: 0
Duplicate ResponseIDs: 0

```

EUREF FUTURE-CHOICE INCENTIVES

```

-----
Respondents: 120
Factors: 13
Missing values: 0
Duplicate ResponseIDs: 0

```

RESPONDENT ID COMPARISON

```

-----
Respondents in both datasets: 120
Only in direct-booking dataset: 0
Only in future-choice dataset: 0

```

```

# =====
# RQ3 - CRM LEAD-SOURCE PERFORMANCE SUMMARY
# =====

```

```

# Use the real-event RQ1 dataset

```

```

rq3_crm = df_rq1.copy()

```

```

# Summarise each lead source

```

```

crm_summary = (
    rq3_crm
    .groupby("Marketing_Measure")["IsCancelled"]
    .agg(
        Total_Events="count",
        Cancelled="sum"
    )
    .reset_index()
)

```

```

# Calculate successful/non-cancelled events

```

```

crm_summary["Not_Cancelled"] = (
    crm_summary["Total_Events"] -
    crm_summary["Cancelled"]
)

# Calculate rates
crm_summary["Cancellation_Rate_%"] = (
    crm_summary["Cancelled"] /
    crm_summary["Total_Events"] * 100
)

crm_summary["Booking_Rate_%"] = (
    crm_summary["Not_Cancelled"] /
    crm_summary["Total_Events"] * 100
)

# Calculate contribution to all real events
crm_summary["Event_Share_%"] = (
    crm_summary["Total_Events"] /
    crm_summary["Total_Events"].sum() * 100
)

# Sort by event volume
crm_summary = crm_summary.sort_values(
    "Total_Events",
    ascending=False
)

print("=" * 80)
print("RQ3 – CRM LEAD–SOURCE PERFORMANCE")
print("=" * 80)

print(
    crm_summary.round(2).to_string(index=False)
)

print("\nTotal real events:", crm_summary["Total_Events"].sum())

```

```

=====
RQ3 – CRM LEAD–SOURCE PERFORMANCE
=====

```

Marketing_Measure	Total_Events	Cancelled	Not_Cancelled	Cancellation_Rate_%	Booking_Rate_%	Event_Share_%
Recurring	845	497	348	58.82	41.18	33.51
Campus Tenant	575	188	387	32.70	67.30	22.80
Digital	398	310	88	77.89	22.11	15.78
Agency	249	217	32	87.15	12.85	9.87
Network	235	130	105	55.32	44.68	9.32
Internal	104	28	76	26.92	73.08	4.12
Industry Association	63	17	46	26.98	73.02	2.50
Venue Portal	45	32	13	71.11	28.89	1.78
Operational	8	2	6	25.00	75.00	0.32

Total real events: 2522

```
# =====
# RQ3 – COMBINE CUSTOMER PRIORITIES
# =====

# RQ2: Venue selection factors
rq2_priority = selection_summary.copy()

rq2_priority["Source"] = "Venue Selection"

rq2_priority = rq2_priority.rename(
    columns={
        "Factor": "Customer_Need",
        "Selection_Rate_%": "Selection_Rate"
    }
)

rq2_priority = rq2_priority[
    ["Customer_Need", "Selection_Rate", "Source"]
]

# RQ3: Direct-booking incentives
direct_priority = direct_summary.copy()

direct_priority["Source"] = "Direct Booking Incentive"

direct_priority = direct_priority.rename(
    columns={
        "Factor": "Customer_Need",
        "Selection_Rate_%": "Selection_Rate"
    }
)

direct_priority = direct_priority[
    ["Customer_Need", "Selection_Rate", "Source"]
]

# RQ3: Future EUREF incentives
future_priority = future_summary.copy()

future_priority["Source"] = "Future EUREF Choice"

future_priority = future_priority.rename(
    columns={
        "Factor": "Customer_Need",
        "Selection_Rate_%": "Selection_Rate"
    }
)
```

```

    }
)

future_priority = future_priority[
    ["Customer_Need", "Selection_Rate", "Source"]
]

# Combine all customer evidence
customer_priorities = pd.concat(
    [
        rq2_priority,
        direct_priority,
        future_priority
    ],
    ignore_index=True
)

# Sort by selection rate
customer_priorities = customer_priorities.sort_values(
    "Selection_Rate",
    ascending=False
)

print("=" * 80)
print("RQ3 – CUSTOMER PRIORITY EVIDENCE")
print("=" * 80)

print(
    customer_priorities.round(2).to_string(index=False)
)

```

```

=====
RQ3 – CUSTOMER PRIORITY EVIDENCE
=====

```

Customer_Need	Selection_Rate	Source
Location & accessibility	70.83	Venue Selection
Transparent pricing	53.33	Direct Booking Incentive
Venue capacity	52.50	Venue Selection
Faster response times	51.67	Direct Booking Incentive
Value for money	48.33	Venue Selection
More competitive pricing	45.83	Future EUREF Choice
Technical/AV equipment	44.17	Venue Selection
Fast communication & response time	40.83	Venue Selection
Faster response times	40.83	Future EUREF Choice
Simpler enquiry/booking process	37.50	Direct Booking Incentive
Dedicated point of contact	36.67	Direct Booking Incentive
Simpler enquiry/booking process	34.17	Future EUREF Choice

Tenant discounts	33.33	Future EUREF Choice
Competitive pricing	33.33	Direct Booking Incentive
Strong venue expertise	32.50	Direct Booking Incentive
Better online information	32.50	Direct Booking Incentive
Better communication on available spaces	30.83	Future EUREF Choice
Better technical/event support	30.00	Direct Booking Incentive
Flexible event spaces	30.00	Venue Selection
Clearer pricing information	29.17	Future EUREF Choice
Dedicated point of contact	28.33	Future EUREF Choice
More flexible contracts	28.33	Future EUREF Choice
Customised event packages	28.33	Direct Booking Incentive
Sustainability features	26.67	Venue Selection
Customised event packages	26.67	Future EUREF Choice
Venue reputation	22.50	Venue Selection
Better technical/hybrid event options	22.50	Future EUREF Choice
Past experience with venue	21.67	Venue Selection
Existing relationship with venue	20.83	Direct Booking Incentive
Stronger sustainability packages	16.67	Future EUREF Choice
More information on event services	11.67	Future EUREF Choice
Other	8.33	Future EUREF Choice

```
# =====
# RQ3 – LEAD-SOURCE STRATEGIC PERFORMANCE CLASSIFICATION
# =====

# Start from the CRM summary already created
strategic_sources = crm_summary.copy()

# Calculate median thresholds
volume_median = strategic_sources["Total_Events"].median()
booking_median = strategic_sources["Booking_Rate_%"].median()

print("=" * 80)
print("RQ3 – STRATEGIC LEAD-SOURCE ANALYSIS")
print("=" * 80)

print(f"Median event volume: {volume_median:.1f}")
print(f"Median booking rate: {booking_median:.2f}%")

# -----
# Classify each lead source
# -----

def classify_source(row):
    high_volume = row["Total_Events"] >= volume_median
    high_booking = row["Booking_Rate_%"] >= booking_median

    if high_volume and high_booking:
        return "High Volume / High Booking Rate"
    elif high_volume and not high_booking:
        return "High Volume / Low Booking Rate"
```

```

elif not high_volume and high_booking:
    return "Low Volume / High Booking Rate"
else:
    return "Low Volume / Low Booking Rate"

strategic_sources["Strategic_Category"] = (
    strategic_sources.apply(classify_source, axis=1)
)

# -----
# Display results
# -----

strategic_sources = strategic_sources.sort_values(
    "Booking_Rate_%",
    ascending=False
)

print("\nLEAD-SOURCE STRATEGIC CLASSIFICATION")
print("-" * 80)

print(
    strategic_sources[
        [
            "Marketing_Measure",
            "Total_Events",
            "Booking_Rate_%",
            "Cancellation_Rate_%",
            "Event_Share_%",
            "Strategic_Category"
        ]
    ].round(2).to_string(index=False)
)

```

=====

RQ3 – STRATEGIC LEAD-SOURCE ANALYSIS

=====

Median event volume: 235.0
 Median booking rate: 44.68%

LEAD-SOURCE STRATEGIC CLASSIFICATION

Marketing_Measure	Total_Events	Booking_Rate_%	Cancellation_Rate_%	Event_Share_%	Strategic_Category
Operational	8	75.00	25.00	0.32	Low Volume / High Booking Rate
Internal	104	73.08	26.92	4.12	Low Volume / High Booking Rate
Industry Association	63	73.02	26.98	2.50	Low Volume / High Booking Rate
Campus Tenant	575	67.30	32.70	22.80	High Volume / High Booking Rate
Network	235	44.68	55.32	9.32	High Volume / High Booking Rate
Recurring	845	41.18	58.82	33.51	High Volume / Low Booking Rate
Venue Portal	45	28.89	71.11	1.78	Low Volume / Low Booking Rate

Digital	398	22.11	77.89	15.78	High Volume / Low Booking Rate
Agency	249	12.85	87.15	9.87	High Volume / Low Booking Rate

```
# =====
# RQ3 - INTEGRATED EVIDENCE MATRIX
# CRM + CUSTOMER SURVEY
# =====

# -----
# 1. Key CRM lead-source findings
# -----

crm_rq3 = crm_summary[
    [
        "Marketing_Measure",
        "Total_Events",
        "Booking_Rate_%",
        "Cancellation_Rate_%",
        "Event_Share%"
    ]
].copy()

# -----
# 2. Select the most important customer requirements
# -----

# Top venue-selection factors
top_venue_needs = (
    selection_summary
    .sort_values("Selection_Rate%", ascending=False)
    .head(5)
    .copy()
)

# Top direct-booking incentives
top_direct_incentives = (
    direct_summary
    .sort_values("Selection_Rate%", ascending=False)
    .head(5)
    .copy()
)

# Top future EUREF incentives
top_future_incentives = (
    future_summary
    .sort_values("Selection_Rate%", ascending=False)
    .head(5)
    .copy()
)
```

```

# -----
# 3. Display integrated evidence
# -----

print("=" * 90)
print("RQ3 – INTEGRATED EVIDENCE")
print("=" * 90)

print("\nCRM LEAD–SOURCE PERFORMANCE")
print("-" * 90)

print(
    crm_rq3.round(2).to_string(index=False)
)

print("\nTOP CUSTOMER VENUE–SELECTION NEEDS")
print("-" * 90)

print(
    top_venue_needs[
        ["Factor", "Selected", "Selection_Rate_%"]
    ].round(2).to_string(index=False)
)

print("\nTOP DIRECT–BOOKING INCENTIVES")
print("-" * 90)

print(
    top_direct_incentives[
        ["Factor", "Selected", "Selection_Rate_%"]
    ].round(2).to_string(index=False)
)

print("\nTOP FUTURE EUREF INCENTIVES")
print("-" * 90)

print(
    top_future_incentives[
        ["Factor", "Selected", "Selection_Rate_%"]
    ].round(2).to_string(index=False)
)

```

```

=====
RQ3 – INTEGRATED EVIDENCE
=====

```

```

CRM LEAD–SOURCE PERFORMANCE
-----

```

Marketing_Measure	Total_Events	Booking_Rate_%	Cancellation_Rate_%	Event_Share_%
Recurring	845	41.18	58.82	33.51

Campus Tenant	575	67.30	32.70	22.80
Digital	398	22.11	77.89	15.78
Agency	249	12.85	87.15	9.87
Network	235	44.68	55.32	9.32
Internal	104	73.08	26.92	4.12
Industry Association	63	73.02	26.98	2.50
Venue Portal	45	28.89	71.11	1.78
Operational	8	75.00	25.00	0.32

TOP CUSTOMER VENUE-SELECTION NEEDS

Factor	Selected	Selection_Rate_%
Location & accessibility	85	70.83
Venue capacity	63	52.50
Value for money	58	48.33
Technical/AV equipment	53	44.17
Fast communication & response time	49	40.83

TOP DIRECT-BOOKING INCENTIVES

Factor	Selected	Selection_Rate_%
Transparent pricing	64	53.33
Faster response times	62	51.67
Simpler enquiry/booking process	45	37.50
Dedicated point of contact	44	36.67
Competitive pricing	40	33.33

TOP FUTURE EUREF INCENTIVES

Factor	Selected	Selection_Rate_%
More competitive pricing	55	45.83
Faster response times	49	40.83
Simpler enquiry/booking process	41	34.17
Tenant discounts	40	33.33
Better communication on available spaces	37	30.83